MABEL Teleoperation Data Collection: A Whole-Body Multimodal Logging Pipeline for Imitation Learning

MABEL Project

June 10, 2026

Abstract

We describe the data-collection subsystem that records demonstrations for imitation learning on MABEL, a mobile bimanual robot (3-module swerve base, cascaded lift, leaning torso, 3-DOF neck, two 7-DOF arms with dexterous hands). An operator teleoperates the robot through an Apple Vision Pro driving the shared whole-body controller, and logs a fully synchronized, multimodal episode: all seven on-board cameras (RGB + metric depth), a 360-ray planar LiDAR, every joint's position/velocity/torque, the floating-base pose and twist, the end-effector and head poses, and the *target command for every actuator* (`data.ctrl`)—the policy's action label. The same schema and recorder back both the MuJoCo simulator and the physical robot, so policies transfer between them without remapping observations or actions. We give the data model and its formal state/action definitions, the timing and depth-encoding equations, and a reproducible benchmark: at 480×270 the full seven-camera configuration produces 225 KB/frame and is render-bound at 1.9 Hz on a CPU-only laptop, while a wrist-only configuration runs at 6.1 Hz; on a GPU offscreen context the target 15 Hz is recovered.

Keywords: imitation learning, teleoperation, Apple Vision Pro, mobile manipulation, whole-body control, multimodal logging, MuJoCo, sim-to-real.

1 Overview

Learning visuomotor policies for a whole-body mobile manipulator requires demonstrations that capture *everything the policy might condition on or predict*—not just a wrist camera and gripper width as in a fixed-arm tabletop setup. MABEL adds a mobile base, a lift/torso reach mechanism, an actively controlled neck, and two dexterous hands, so a demonstration must record the full sensor suite and the full action vector across the body.

This subsystem (directory `data_collection/`) is a generalization of the one-arm/one-hand tabletop logger in `dexterous_hand/data_collection` to MABEL's entire body. The operator teleoperates through the same `TeleopController` used by the live viewer and the Vision Pro bridge [3], so the demonstrated motion is generated by exactly the control law a learned policy will later replace—there is no separate scripted data source to drift from deployment. The design follows the now-standard teleoperation-for-imitation recipe of ALOHA/ACT and Mobile ALOHA [1, 2], the relative-pose (clutch) end-effector mapping of UMI [4], and the multi-view, multimodal episode format of large robot datasets such as DROID and Open X-Embodiment [5, 6].

The code is organized so the only platform-specific part is a thin *state provider*:

- `config.py` — recording configuration; joint/camera/sensor names are pulled from the MuJoCo model and the shared controller config (`controller/mabel/config.py`), never hard-coded.

- `providers.py` — `SimStateProvider` (reads `MjData` and renders the cameras) and `RealStateProvider` (hardware skeleton); both emit the *identical* observation/action dictionary.
- `recorder.py` — `EpisodeRecorder`: accumulates frames and writes one HDF5 file per episode on a background thread.
- `collect_sim.py` / `collect_real.py` — the operator loops.
- `inspect_episode.py`, `visualize_dataset.py`, `merge_dataset.py`, `tools/bench_collection.py` — tooling.

2 Method

2.1 Robot and notation

MABEL's generalized configuration is $q \in \mathbb{R}^{n_q}$ with $n_q = 81$ and velocity $\dot{q} \in \mathbb{R}^{n_v}$, $n_v = 78$ (the discrepancy is the floating-base quaternion: 7 position vs. 6 velocity coordinates). The robot is driven by $n_u = 59$ actuators, grouped as

$$\underbrace{6}_{\text{base (3 steer + 3 drive)}} + \underbrace{1}_{\text{lift}} + \underbrace{1}_{\text{torso}} + \underbrace{3}_{\text{neck}} + \underbrace{2 \times 8}_{\text{arms (7+wrist)}} + \underbrace{2 \times 16}_{\text{hands}} = 59. \tag{1}$$

All position actuators take a target joint angle; the three base "drive" actuators take a wheel angular velocity and the three "steer" actuators take a module angle. The command vector $u =$ `data.ctrl` $\in \mathbb{R}^{59}$ is therefore the complete instantaneous *target* for the whole body.

2.2 Action: the whole-body command

We log the action as the controller's command vector at the logging instant,

$$a_t \;=\; u_t \;=\; \texttt{data.ctrl}\big|_t \;\in\; \mathbb{R}^{59}, \tag{2}$$

indexed identically to the actuator list of (1), whose names and group membership are stored in the episode metadata so downstream code can slice out, e.g., the left-arm sub-action without positional guesswork. We additionally log the *operator intent* produced by the controller—the bimanual palm SE(3) targets, the scalar grasp per hand, the base twist command (v_x, v_y, ω), the gaze look-at point, and the discrete operator mode (manipulation vs. navigation). These are not required to imitate a_t but make the data usable for higher-level or hierarchical policies, and for debugging the retargeting.

2.3 Observation: the full sensor state

The per-frame observation o_t concatenates, by modality:

- **Vision.** Up to seven cameras (two wrists, ZED stereo pair on the head, a head/body camera, and front/back body cameras). Each logs an RGB frame and, optionally, a metric depth frame.
- **LiDAR.** A 360-ray planar scan $\ell \in \mathbb{R}^{360}$ at 1° resolution (Slamtec RPLIDAR A3 model), in metres.
- **Proprioception.** Per-actuated-joint position, velocity, and applied torque, $q^{\mathrm{a}}, \dot{q}^{\mathrm{a}}, \tau^{\mathrm{a}} \in \mathbb{R}^{59}$, aligned with the action index; plus the full $q \in \mathbb{R}^{81}$, $\dot{q} \in \mathbb{R}^{78}$ as unambiguous ground truth in simulation.
- **Body kinematics.** World poses (position + quaternion) of the chassis, head, and both palms, and the measured base twist (v_x, v_y, ω_z) taken from the chassis spatial velocity.

The joint torque uses the applied actuator force τ^{a} = `data.actuator_force`; on hardware this is the commanded/measured motor torque from the bus, keeping the field meaningful across sim and real.

2.4 Depth encoding

Depth is rendered as a float metric buffer $d(p)$ (metres) per pixel p and stored as `uint16` millimetres after clipping to a maximum range $d_{\max} = 8\,\mathrm{m}$:

$$d_{16}(p) \;=\; \mathrm{round}\Big(\mathrm{clip}\big(d(p),\,0,\,d_{\max}\big)\cdot s\Big), \qquad s = 1000\,\mathrm{mm/m}, \tag{3}$$

which is lossless to the millimetre over $[0, 8\,\mathrm{m}]$ and fits the `uint16` range $[0, 8000] \subset [0, 65535]$. RGB frames are JPEG-encoded (quality 90) and stored as variable-length byte blobs—near-lossless for training, and an order of magnitude smaller than raw frames (Table 1).

2.5 Timing and rate decoupling

Physics integrates at $f_{\mathrm{phys}} = 1/\Delta t_{\mathrm{phys}} = 500\,\mathrm{Hz}$ (`model.opt.timestep` = 2 ms). The control substep `step()` runs every physics tick, but logging is decoupled at a target rate $f_{\log}$ (default 15 Hz): a frame is captured when wall-clock time crosses the next $1/f_{\log}$ boundary,

$$t^{\log}_{k+1} \;=\; \max\big(t^{\log}_{k} + 1/f_{\log},\; t_{\mathrm{now}}\big), \tag{4}$$

where the max prevents a backlog from accumulating when a heavy render makes one frame late (the logger drops the missed slot rather than bursting). Each frame stores both the simulation time and the wall time, so the achieved rate and any jitter are recoverable post hoc.

2.6 Episode segmentation

The operator owns episode boundaries. `SPACE` toggles recording: a falling edge stops the current episode and queues it for saving; a rising edge resets the robot to its home configuration, re-randomizes the manipuland pose, and begins a new buffer. `X` discards an in-progress episode; `R` resets/re-randomizes without changing the recording state. Saving runs on a background thread so a multi-hundred-frame episode never stalls teleoperation. Episode files are numbered contiguously and numbering auto-resumes from disk, so a session can be paused and restarted without overwriting.

3 Experiments

All numbers are produced from scratch by `tools/bench_collection.py`, which runs the entire pipeline headlessly with the synthetic (mock) Vision Pro source, records a 60-frame episode at a fixed seed, and measures wall-clock render time and the on-disk byte breakdown per modality. The diagnostic figures (Fig. 1–Fig. 3) are regenerated from that same episode by `visualize_dataset.py`. Measurements are on a CPU-only laptop using MuJoCo's software (CGL) offscreen renderer—deliberately the worst case for the rendering bottleneck.

Table 1 sweeps the dominant cost knob, the number of cameras. Rendering and JPEG/depth encoding dominate wall time (the buffer append is 0.02 ms/frame, negligible); the sustainable log rate is therefore set by the render cost. With all seven cameras the pipeline is render-bound at 1.9 Hz on this machine—below the 15 Hz target—whereas a wrist-only pair reaches 6.1 Hz. Image

Table 1: Collection benchmark at 480×270 per camera, 60 frames, seed 0, CPU/software renderer. "Rate" is the render-bound sustainable logging rate ($1/\overline{t_{\text{render}}}$). Image bytes dominate; state/action are negligible.

Configuration	Cameras (rgb/depth)	Render/frame (ms)	Rate (Hz)	Size/frame (KB)	RGB (%)	Depth (%)
All on-board	7 / 5	535 ± 121	1.9	225	50.8	45.0
Wrist-only	2 / 2	164 ± 45	6.1	101	34.4	60.1

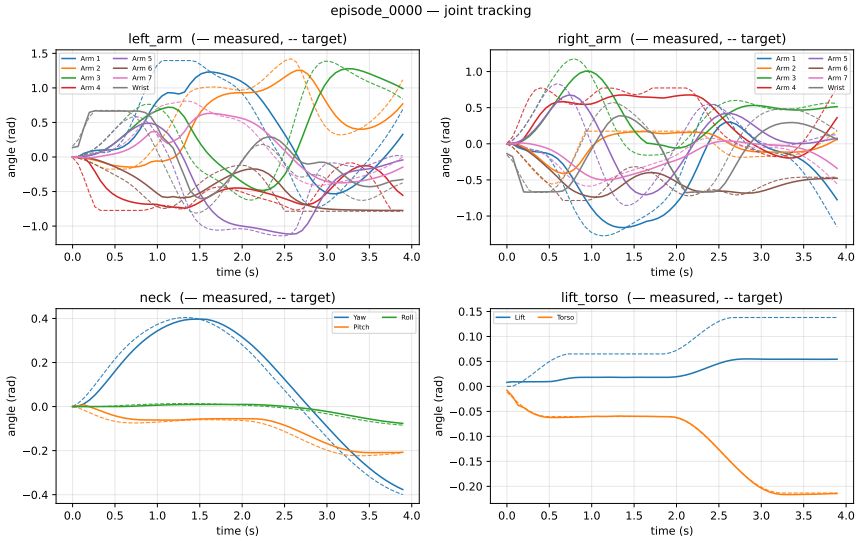


Figure 1: Joint tracking for a recorded episode: commanded target (`data.ctrl`, dashed) vs. measured joint angle (solid) for the left/right arms, neck, and lift/torso. The gap between the curves is the controller's tracking error and is exactly what an imitation policy must reproduce when it predicts the action.

data is ∼96 % of episode size, split roughly evenly between RGB (JPEG) and depth (gzipped `uint16`); all of proprioception, LiDAR, body kinematics, and the action together are under 1 %.

4 Implementation and Usage

4.1 Running collection in simulation

1. Start the Vision Pro publisher (in `dexterous_hand/vision_pro`):
   `python connect_full_avp.py --room <ROOM_ID>`.
2. Collect (episodes land in `datasets/<name>/episode_XXXX.h5`):
   `python collect_sim.py --name pick_cube_v1 --task "pick the blue cube"`.
   Use `--mock` for a headless synthetic source, or `--addr` for a remote publisher.

Operator keys: `C` (re)calibrate the clutch, `M` toggle manipulation/navigation, `B` toggle lazy base-follow, `SPACE` start/stop+save, `X` discard, `R` reset+randomize, `Q`/`ESC` quit. Fig. 1–Fig. 3 and Table 1 are regenerated by

`python tools/bench_collection.py --frames 60 --seed 0`.

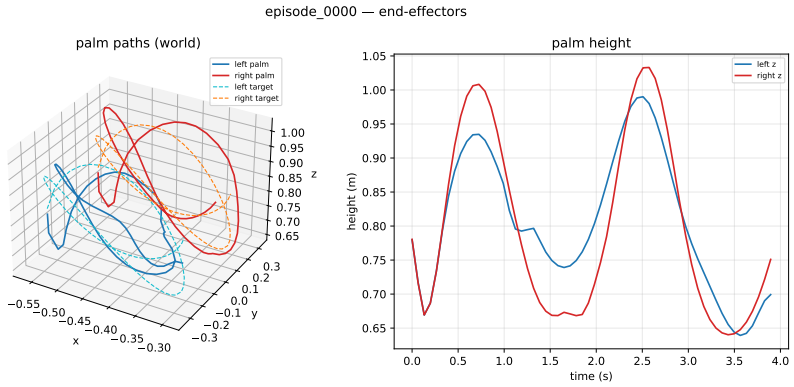


Figure 2: End-effector logging: left/right palm world paths (with the operator's SE(3) targets overlaid) and palm height over time. Confirms the operator-intent and measured-pose channels are captured and consistent.

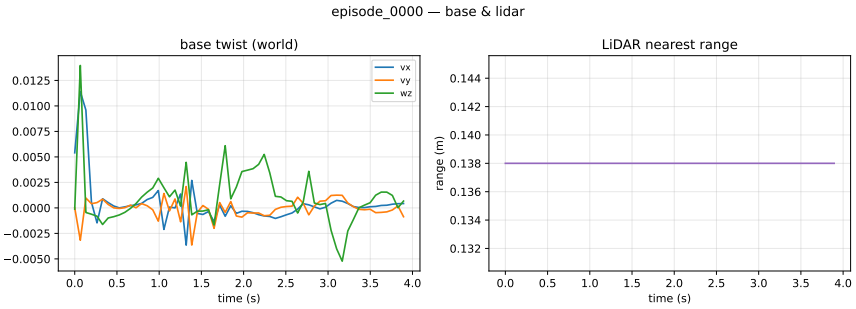


Figure 3: Mobile-base and LiDAR channels: measured base twist (v_x, v_y, ω_z) and the nearest LiDAR return over the episode—the navigation-relevant state a fixed-arm logger would omit.

4.2 Episode format

One HDF5 file per episode, every dataset time-major (axis 0 = frame). RGB streams are variable-length `uint8` JPEG blobs; depth is `uint16` per (3); all other arrays are gzip-compressed. The actuator names and ordering, camera list, units, and $n_q/n_v/n_u$ are written to the file attributes (`schema_json`) and a sibling `dataset_meta.json`.

4.3 The sim-to-real path

`collect_real.py` and `RealStateProvider` mirror the sim entry points with identical keys and schema. On hardware the operator drives the *same* `TeleopController` against a MuJoCo shadow model whose configuration is kept synced to the encoders each tick; the controller's `data.ctrl` is streamed to the motor bus, and the provider fills the observation from the real cameras, LiDAR, and bus telemetry (scaling depth by (3) identically). Because the key layout is shared, a policy trained on simulation episodes consumes hardware episodes—and produces hardware actions—without any remapping.

5 Limitations

- **Render-bound on CPU.** With seven cameras the software renderer sustains only $\sim$2 Hz (Table 1); reaching the 15 Hz target needs a GPU offscreen context (EGL) or a reduced camera set. The rate-decoupling of (4) keeps the data self-consistent (true timestamps) but the effective demonstration rate is what the hardware achieves, not the configured $f_{\log}$.
- **Default depth subset.** To bound episode size, depth defaults on for five views and off for the rear/secondary stereo view; this is a configuration choice in `config.CAMERAS`, not a hard limit.
- **Mock validation only here.** The numbers above come from the synthetic Vision Pro source; the geometry and motion of mock frames are simple, so absolute image-compression sizes will rise on cluttered real scenes (the relative modality split and the render-bound conclusion hold). Real-operator dataset statistics are reported once collected.
- **Real provider is a skeleton.** `RealStateProvider` defines the contract but its hardware I/O hooks must be wired to MABEL's drivers; the sim path is the only one exercised end-to-end in this document.

References

[1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ALOHA/ACT)," *Robotics: Science and Systems (RSS)*, 2023.

[2] Z. Fu, T. Z. Zhao, and C. Finn, "Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation," *Conference on Robot Learning (CoRL)*, 2024.

[3] X. Cheng, J. Li, S. Yang, G. Yang, and X. Wang, "Open-TeleVision: Teleoperation with Immersive Active Visual Feedback," *Conference on Robot Learning (CoRL)*, 2024.

[4] C. Chi et al., "Universal Manipulation Interface: In-The-Wild Robot Teaching Without In-The-Wild Robots," *Robotics: Science and Systems (RSS)*, 2024.

[5] A. Khazatsky et al., "DROID: A Large-Scale In-the-Wild Robot Manipulation Dataset," *Robotics: Science and Systems (RSS)*, 2024.

[6] Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic Learning Datasets and RT-X Models," *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2024.

[7] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A Physics Engine for Model-Based Control," *IEEE/RSJ IROS*, 2012.